

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ

Кафедра програмної інженерії

Звіт
до лабораторної роботи № 3
«Модульне тестування програмного коду (Unit Testing)»
з дисципліни
«Основи програмної інженерії»

Виконали: Куцевич Анна Миколаївна,
Рижова Марія Володимирівна,
студентки групи ПЗПІ-25-1.
Викладач: В'ячеслав Олександрович
Гребенюк

Харків 2026

1. Мета роботи

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

2. Вихідний код реалізованого модуля з коментарями.

program_code\index.html

```
<!doctype html>
<html lang="uk">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>AnswerBetter</title>
  </head>
  <body>
    <h1>AnswerBetter</h1>

    <label for="message">Введіть текст:</label><br />
    <textarea
      id="message"
      rows="8"
      cols="60"
      placeholder="Введіть повідомлення"
    ></textarea>
    <br /><br />

    <label for="tone">Оберіть стиль:</label><br />
    <select id="tone">
      <option value="formal">Формальний</option>
```

```

    <option value="friendly">Дружній</option>
    <option value="professional">Професійний</option>
</select>
<br /><br />

<button id="generateBtn">Покращити повідомлення</button>
<button id="copyBtn">Копіювати результат</button>

<h2>Результат</h2>
<div id="result"></div>

<h2>Історія відповідей</h2>
<ul id="history"></ul>

<script src="program_code/main.js"></script>
</body>
</html>

```

program_code\main.js

```

const messageInput = document.getElementById("message");
const toneSelect = document.getElementById("tone");
const generateBtn = document.getElementById("generateBtn");
const copyBtn = document.getElementById("copyBtn");
const resultBlock = document.getElementById("result");
const historyList = document.getElementById("history");

const history = [];

function inputText(text) {
  if (typeof text !== "string") {
    throw new TypeError("Text must be a string");
  }

  const trimmed = text.trim();

  if (trimmed.length === 0) {
    throw new Error("Text cannot be empty");
  }
}

```

```

    if (trimmed.length > 500) {
        throw new Error("Text is too long");
    }

    return trimmed;
}

function formatPunctuation(text) {
    let result = text

        .trim()
        .replace(/\s*,\s*/g, ", ")
        .replace(/\s*\.\s*/g, ". ")
        .replace(/\s*!\s*/g, "! ")
        .replace(/\s*\?\s*/g, "? ")
        .replace(/\s+/g, " ")
        .trim()
        .toLowerCase();

    result = result.replace(/^(^|[\.!?]\s+)([a-zA-яёиїєґ])/giu, (match,
start, letter) => {
        return start + letter.toUpperCase();
    });

    result = result.replace(/, \s*([a-zA-яёиїєґ])/giu, (match, letter)
=> {
        return ", " + letter.toLowerCase();
    });

    return result;
}

function generateMessage(text, tone) {
    const validText = inputText(text);
    let result = "";

    if (tone === "formal") {
        result = `Добрый день, ${validText}`;
    } else if (tone === "friendly") {

```

```
        result = `Привіт! ${validText}`;
    } else {
        result = `Вітаю, ${validText}`;
    }

    return formatPunctuation(result);
}

function saveToHistory(result) {
    history.push(result);
    renderHistory();
}

function renderHistory() {
    historyList.innerHTML = "";

    history.forEach((item) => {
        const li = document.createElement("li");
        li.textContent = item;
        historyList.appendChild(li);
    });
}

function copyResult(text) {
    if (!text || text.trim().length === 0) {
        throw new Error("Немає тексту для копіювання");
    }

    navigator.clipboard.writeText(text);
}

generateBtn.addEventListener("click", () => {
    try {
        const text = messageInput.value;
        const tone = toneSelect.value;

        const result = generateMessage(text, tone);
        resultBlock.textContent = result;
        saveToHistory(result);
    }
});
```

```

    } catch (error) {
        resultBlock.textContent = error.message;
    }
});

copyBtn.addEventListener("click", () => {
    try {
        copyResult(resultBlock.textContent);
        alert("Результат скопійовано");
    } catch (error) {
        resultBlock.textContent = error.message;
    }
});

module.exports = {
    history,
    inputText,
    formatPunctuation,
    generateMessage,
    saveToHistory,
    copyResult
};

```

3. Таблиця проєктування тестів: «Тест-кейс | Вхідні дані | Очікуваний результат | Техніка (EP/BVA) | Статус (pass/fail)».

docs\lab3\ep-bva-kutsevych.md

EP та BVA – Куцевич

| Тест-кейс | Вхідні дані | Очікуваний результат | Техніка | Статус |

|---|---|---|---|---

| TC-01 | inputText(" hello ") | "hello" | EP, позитивний | pass |

| TC-02 | inputText(123) | TypeError | EP, негативний | pass |

| TC-03 | inputText(" ") | Error "Text cannot be empty" | EP, негативний |

pass |

| TC-06 | formatPunctuation("ти вивчив фігму. тепер вмієш") | "Ти вивчив фігму. Тепер вмієш" | EP | pass |

| TC-08 | generateMessage("я вивчив фігму", "formal") | "Добрий день, я вивчив фігму" | EP, позитивний | pass |

| TC-09 | generateMessage("я вивчив фігму", "friendly") | "Привіт! Я вивчив фігму" | EP, позитивний | pass |

EP та BVA – Куцевич

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
TC-01	inputText(" hello ")	"hello"	EP, позитивний	pass
TC-02	inputText(123)	TypeError	EP, негативний	pass
TC-03	inputText(" ")	Error "Text cannot be empty"	EP, негативний	pass
TC-06	formatPunctuation("ти вивчив фігму. тепер вмієш")	"Ти вивчив фігму. Тепер вмієш"	EP	pass
TC-08	generateMessage("я вивчив фігму", "formal")	"Добрий день, я вивчив фігму"	EP, позитивний	pass
TC-09	generateMessage("я вивчив фігму", "friendly")	"Привіт! Я вивчив фігму"	EP, позитивний	pass

docs\lab3\ep-bva-ryzhova.md

EP та BVA – Рижова

| Тест-кейс | Вхідні дані | Очікуваний результат | Техніка | Статус |

|---|---|---|---|---|

| TC-04 | inputText("a".repeat(500)) | рядок довжиною 500 | BVA | pass |

| TC-05 | inputText("a".repeat(501)) | Error "Text is too long" | BVA | pass |

| TC-07 | formatPunctuation("привіт, СВІТ") | "Привіт, світ" | EP | pass |

| TC-10 | generateMessage("я вивчив фігму", "professional") | "Вітаю, я вивчив фігму" | EP, позитивний | pass |

| TC-11 | saveToHistory("Тест") | history містить 1 елемент | EP | pass |

| TC-12 | copyResult("") | Error "Немає тексту для копіювання" | EP, негативний | pass |

EP та BVA – Рижова

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
TC-04	inputText("a".repeat(500))	рядок довжиною 500	BVA	pass
TC-05	inputText("a".repeat(501))	Error "Text is too long"	BVA	pass
TC-07	formatPunctuation("привіт, СВІТ")	"Привіт, світ"	EP	pass
TC-10	generateMessage("я вивчив фірму", "professional")	"Вітаю, я вивчив фірму"	EP, позитивний	pass
TC-11	saveToHistory("Тест")	history містить 1 елемент	EP	pass
TC-12	copyResult("")	Error "Немає тексту для копіювання"	EP, негативний	pass

4. Вихідний код тестового набору з коментарями.

program_code\tests\main.test.js

```
let history, inputText, formatPunctuation, generateMessage,
saveToHistory, copyResult;

beforeAll(() => {
  document.body.innerHTML = `
    <input id="message" />
    <select id="tone"></select>
    <button id="generateBtn"></button>
    <button id="copyBtn"></button>
    <div id="result"></div>
    <ul id="history"></ul>
  `;
  ({ history, inputText, formatPunctuation, generateMessage,
saveToHistory, copyResult } = require("../main"));
});

describe("Куцєвич tests", () => {
  beforeEach(() => {
    history.length = 0;
  });

  test("TC-01 inputText trims text", () => {
    const text = "  hello  ";

    const result = inputText(text);
```



```
    expect(result).toBe("hello");
  });

  test("TC-02 inputText throws TypeError for non-string", () => {
    const text = 123;
    expect(() => inputText(text)).toThrow(TypeError);
  });

  test("TC-03 inputText throws error for empty text", () => {
    const text = " ";
    expect(() => inputText(text)).toThrow("Text cannot be empty");
  });

  test("TC-06 formatPunctuation capitalizes after dot", () => {
    const text = "ти вивчив фірму. тепер вмієш";
    const result = formatPunctuation(text);
    expect(result).toBe("Ти вивчив фірму. Тепер вмієш");
  });

  test("TC-08 generateMessage creates formal message", () => {
    const text = "я вивчив фірму";
    const tone = "formal";
    const result = generateMessage(text, tone);
    expect(result).toBe("Добрий день, я вивчив фірму");
  });

  test("TC-09 generateMessage creates friendly message", () => {
    const text = "я вивчив фірму";
    const tone = "friendly";
    const result = generateMessage(text, tone);
    expect(result).toBe("Привіт! Я вивчив фірму");
  });
});

describe("Рижова tests", () => {
  beforeEach(() => {
```

```
    history.length = 0;

    Object.assign(navigator, {
      clipboard: {
        writeText: jest.fn().mockResolvedValue(undefined),
      },
    });
  });

  test("TC-04 inputText accepts 500 chars", () => {
    const text = "a".repeat(500);
    const result = inputText(text);
    expect(result.length).toBe(500);
  });

  test("TC-05 inputText throws error for 501 chars", () => {
    const text = "a".repeat(501);
    expect(() => inputText(text)).toThrow("Text is too long");
  });

  test("TC-07 formatPunctuation lowercases after comma", () => {
    const text = "привіт, СВІТ";
    const result = formatPunctuation(text);
    expect(result).toBe("Привіт, світ");
  });

  test("TC-10 generateMessage creates professional message", () => {
    const text = "я вивчив фірму";
    const tone = "professional";
    const result = generateMessage(text, tone);
    expect(result).toBe("Вітаю, я вивчив фірму");
  });

  test("TC-11 saveToHistory stores result", () => {
    const result = "Тест";
    saveToHistory(result);
    expect(history).toEqual(["Тест"]);
  });
}
```

```

    test("TC-12 copyResult throws error for empty text", () => {
        const text = "";
        expect(() => copyResult(text)).toThrow("Немає тексту для
копіювання");
    });

    test("TC-13 copyResult writes text to clipboard", async () => {
        const text = "Скопіюй мене";
        await copyResult(text);

expect(navigator.clipboard.writeText).toHaveBeenCalledWith(text);
    });

});

    test("extra click on generateBtn shows generated result and saves
history", () => {
        document.body.innerHTML = `
            <textarea id="message"></textarea>
            <select id="tone">
                <option value="formal">formal</option>
                <option value="friendly">friendly</option>
                <option value="professional">professional</option>
            </select>
            <button id="generateBtn"></button>
            <button id="copyBtn"></button>
            <div id="result"></div>
            <ul id="history"></ul>
        `;

        jest.resetModules();
        const app = require("./main");

        document.getElementById("message").value = "я вивчив фірму";
        document.getElementById("tone").value = "formal";
document.getElementById("generateBtn").click();
expect(document.getElementById("result").textContent)

```

```

        .toBe("Добрий день, я вивчив фірму");

expect(document.getElementById("history").children.length).toBe(1);
});

test("extra click on generateBtn shows error message for empty
input", () => {
    document.body.innerHTML = `
        <textarea id="message"></textarea>
        <select id="tone">
            <option value="formal">formal</option>
        </select>
        <button id="generateBtn"></button>
        <button id="copyBtn"></button>
        <div id="result"></div>
        <ul id="history"></ul>
    `;

    jest.resetModules();
    require("./main");

    document.getElementById("message").value = " ";
    document.getElementById("tone").value = "formal";
    document.getElementById("generateBtn").click();
    expect(document.getElementById("result").textContent)
        .toBe("Text cannot be empty");
});

```

5. Скріншот або HTML-звіт покриття коду з відображенням відсотку line coverage.

```
PS C:\Users\User\OneDrive\Рабочий стол\OPI\annaku.github.io> npm run coverage
```

```
✓ TC-12 copyResult throws error for empty text (3 ms)
```

```
✓ TC-13 copyResult writes text to clipboard (2 ms)
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	92.45	100	90.9	92.45	
main.js	92.45	100	90.9	92.45	102-106

```
Test Suites: 1 passed, 1 total
```

```
Tests: 15 passed, 15 total
```

```
Snapshots: 0 total
```

```
Time: 3.315 s
```

```
Ran all test suites.
```

6. Посилання на Git-репозиторій з кодом модуля та тестами.

Посилання: <https://github.com/annaku-pixel/annaku.github.io.git>

На основі UML-моделі з ЛР 02 (docs/lab2) було створено таблиці проєктування тестів (docs/lab3), на основі яких - розроблено код і написано тести (program_code).

7. Висновки (аналіз отриманих результатів покриття, виявлені проблеми, шляхи поліпшення).

У ході виконання лабораторної роботи було реалізовано програмний модуль для системи AnswerBetter та створено набір модульних тестів для перевірки його основної функціональності.

Для проєктування тестів було використано техніки еквівалентного розбиття та аналізу граничних значень. На їх основі сформовано набір позитивних і негативних тест-кейсів, який дозволяє перевірити як коректну поведінку програми, так і обробку помилкових вхідних даних. Усі модульні тести було реалізовано за допомогою фреймворку Jest із дотриманням патерну AAA.

У результаті виконання лабораторної роботи було здобуто практичні навички написання unit-тестів, аналізу покриття коду та підготовки тестового набору для веб-модуля, реалізованого мовою JavaScript. Також було підготовлено конфігурацію для автоматичного запуску тестів у GitHub Actions, що дає змогу інтегрувати тестування в CI-процес.